UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: AGOSTO - ENERO 2026

DEBER

I. PORTADA

Tema: TAREA 01. Unidad 01 Gestión de datos utilizando vectores en C++
Unidad de Organización Curricular: BÁSICA
Nivel y Paralelo: 3do-B
Alumnos participantes: RODRIGUEZ JIMENEZ DAVID ISRAEL
……..
Asignatura: Estructura de Datos
Docente: Ing. Caiza Caizabuano Jose Ruben Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:
Desarrollar un sistema en C++ que me permita gestionar los pedidos de una cafetería aplicando vectores de estructuras y un menú interactivo

Específicos:

- Analizar la gestión que se aplicara mediante vectores para almacenar registros de pedidos
- Estudiar la gestión aplicada y la estructura aplicada con C++ organizando la información de cada pedido
- Desarrollar un programa de gestión para una cafetería donde se permita el registro, consultar, modificar, eliminar y generar reportes de pedidos

2.2 Instrucciones

Requerimientos del programa
El sistema debe permitir, como mínimo, las siguientes operaciones:

1. Registrar datos
2. Mostrar todos los registros
3. Buscar un elemento por un criterio definido
4. Modificar un registro existente
5. Eliminar un registro
6. Generar un reporte final
7. Salir del sistema

Condiciones obligatorias
Para que el trabajo sea válido, deberá cumplir con lo siguiente:

- Uso obligatorio de vectores
- Desarrollo en C++
- Menú interactivo funcional
- Mínimo 10 registros iniciales
- Datos coherentes con el caso asignado
- Código ordenado y comentado
-

Elementos que debe incluir el programa
El estudiante deberá definir claramente:

- Qué tipo de datos almacenará
- Qué campos tendrá cada registro

- Qué operaciones realizará el sistema
- Qué tipo de vector utilizará:
  - vectores paralelos, o
  - vector de estructuras

Recomendación: Para este nivel, se sugiere utilizar vector de estructuras, ya que permite una mejor organización de la información.

Desarrolle un programa original, comprensible y funcional. Más importante que "entregar un código" es demostrar que comprende cómo gestionar colecciones de datos utilizando vectores y cómo aplicar esta estructura en problemas reales de ingeniería de software.

Entregables:

Codigo fuente (suba el archivo en el github y bloc de notas "deberes")

Informe

2.3 Listado de equipos, materiales y recursos

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☒Plataformas educativas
☒Simuladores y laboratorios virtuales
☐Aplicaciones educativas
☒Recursos audiovisuales
☐Gamificación
☐Inteligencia Artificial
Otros (Especifique): _____

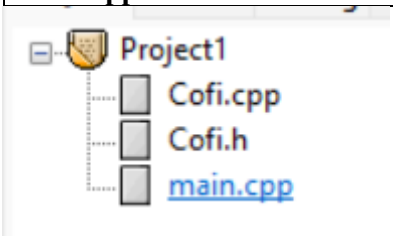
2.4 Actividades por desarrollar

Desarrollar la guía APE en base al formato y las instrucciones brindadas

2.5 Resultados obtenidos

Estructura del Programa

Archivo	Función
Cofi.h	Define la estructura de los datos y declara las funciones
Cofi.cpp	Implementación de las funciones
Main.cpp	Implementación del menú y la ejecución



Estructura del pedido

```
// estructura del pedido
struct Pedido{
    int id;              //identificador del pedido
    string cliente;      //nombre del cliente
    string producto;    // producto que compra
    int cantidad;       // cantidad del producto
    float precio;
};
```

En la estructura del programa exactamente en cada pedido se mostrará el id, cliente, producto, cantidad, precio
El programa utiliza un **vector de estructuras** donde permite almacenar hasta 50 pedidos

```
Pedido pedidos[50]; //vector de estructuras
```

Definición de las funciones
Las funciones están definidas dentro de Cofi.h y su implementación esta en Cofi.cpp

- **Función mostrarPedidos()**
  Este recorre el vector mediante un ciclo for y muestra la información de todos los pedidos almacenados mostrándolos al usuario
  Cofi.h:

```
void mostrarPedidos(Pedido pedidos[], int total);
```

  Cofi.cpp:

```
void mostrarPedidos(Pedido pedidos[], int total){

    for(int i=0;i<total;i++){

        cout<<"\nID: "<<pedidos[i].id<<endl;
        cout<<"Cliente: "<<pedidos[i].cliente<<endl;
        cout<<"Producto: "<<pedidos[i].producto<<endl;
        cout<<"Cantidad: "<<pedidos[i].cantidad<<endl;
        cout<<"Precio: "<<pedidos[i].precio<<endl;
    }
}
```

- **Función buscarPedido()**
  Esta función permite la búsqueda de los pedidos con un vector utilizando un ID, este funciona con un vector en un ciclo for comparando el ID ingresado por el usuario con los del registro si lo encuentra solo devuelve la posición del vector sino devuelve -1
  Cofi.h:

```
int buscarPedido(Pedido pedidos[], int total, int id);
```

  Cofi.cpp:

```
int buscarPedido(Pedido pedidos[], int total, int id){

    //recorre todos los pedidos
    for(int i=0;i<total;i++){
        //busca el pedido del ID
        if(pedidos[i].id==id){
            return i; //devuelve la posicion del pedido
        }
    }
    //POR SI encuentro nada
    return -1;
}
```

- **Función reporteFinal()**
  Esta función calcula el valor total de las ventas funciona recorriendo un vector de pedidos multiplicando la cantidad por el precio a cada pedido y con un acumulador de saca el total de ventas
  Cofi.h:

```
void reporteFinal(Pedido pedidos[], int total);
```

  Cofi.cpp:

```
//CALCULA EL TOTAL DE VENTAS
void reporteFinal(Pedido pedidos[], int total){

    float totalVentas=0;
    //recorre todos los pedidos
    for(int i=0;i<total;i++){
        //multiplica  cantidad por precio
        totalVentas += pedidos[i].cantidad * pedidos[i].precio;
    }
    cout<<"Total de ventas: "<<totalVentas<<endl;
}
```

Menu interactivo functional

Se creo un menú interactivo donde con switch se ordenara de 1 a 7 las opciones.

```
//Menu interactivos
do{

    cout<<"\n===== CAFETERIA =====<<endl;
    cout<<"1 Registrar pedido"<<endl;
    cout<<"2 Mostrar pedidos"<<endl;
    cout<<"3 Buscar pedido"<<endl;
    cout<<"4 Modificar pedido"<<endl;
    cout<<"5 Eliminar pedido"<<endl;
    cout<<"6 Reporte final"<<endl;
    cout<<"7 Salir"<<endl;
    cout<<"Seleccione opcion: ";
    cin>>opcion;

    switch(opcion){
```

Registros inciales (minimo 10)

```
int main(){

    Pedido pedidos[50]; //vector de estructuras
    int total = 10;     //numero de registros iniciales
    int opcion;         //menu
    int id;             //buscador de pedido
    int pos;            //posicion para encontrar el pedido en el vector

    // 10 registros iniciales
    pedidos[0]={1,"David","Cafe",2,1.50};
    pedidos[1]={2,"Estiven","Capuccino",1,2.00};
    pedidos[2]={3,"Jeampierre","Latte",2,2.50};
    pedidos[3]={4,"Pablo","Cafe",3,1.50};
    pedidos[4]={5,"Cesar","Mocaccino",1,2.80};
    pedidos[5]={6,"Diana","Cafe",2,1.50};
    pedidos[6]={7,"Cristian","Latte",1,2.50};
    pedidos[7]={8,"Snaider","Capuccino",2,2.00};
    pedidos[8]={9,"Shirley","Cafe",1,1.50};
    pedidos[9]={10,"Lesly","Latte",2,2.50};
```

Diagrama de clases - Cafetería

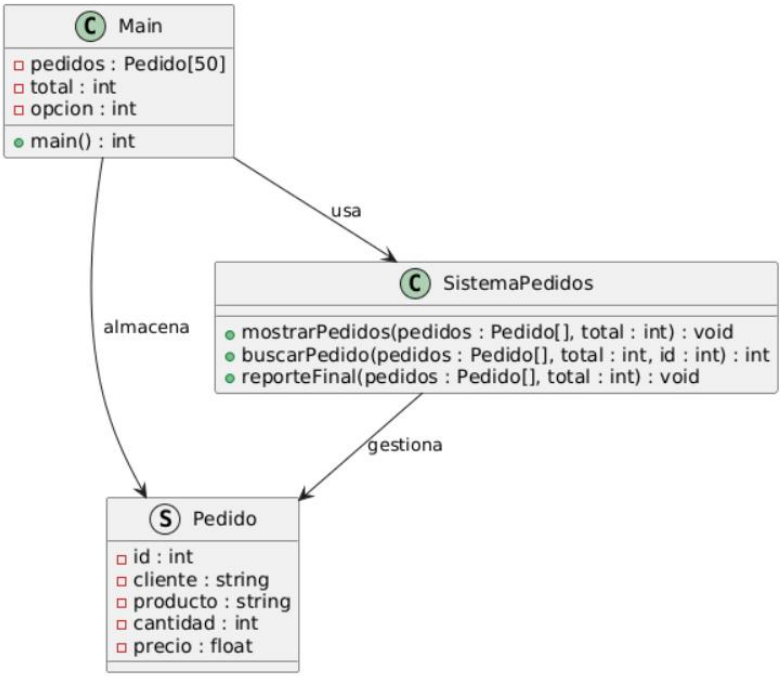


Diagrama de secuencias – Cafetería

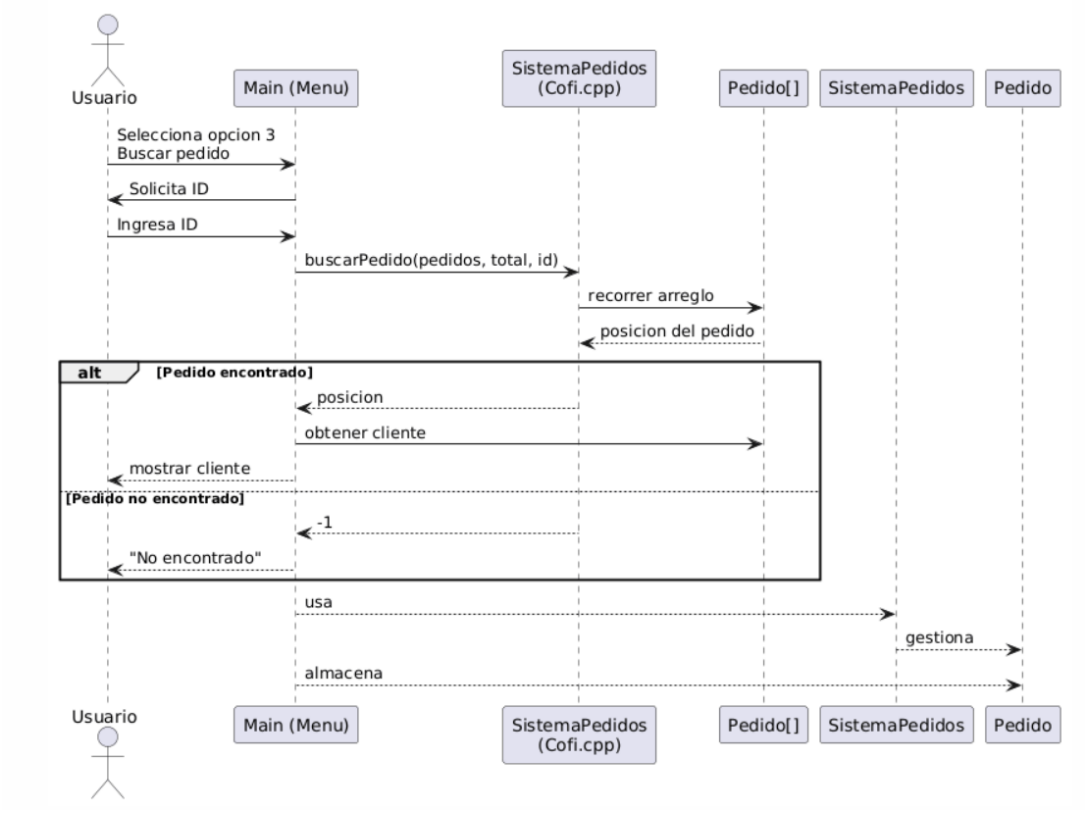
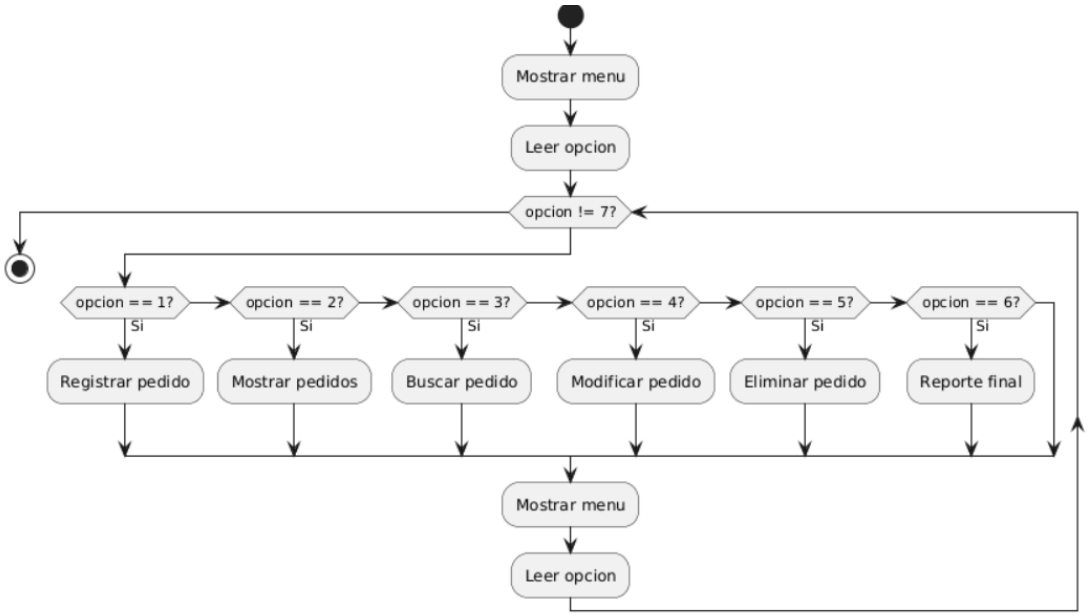


Diagrama de flujo - Cafetería



GitHub: https://github.com/Davu15/Gestion-de-pedidos-de-una-cafeteria.git

2.6 Habilidades blandas empleadas en la práctica

☐ Liderazgo
☐ Trabajo en equipo
☐ Comunicación asertiva

☐ La empatía
☒ Pensamiento crítico
☐ Flexibilidad
☒ La resolución de conflictos
☐ Adaptabilidad
☒ Responsabilidad

2.7 Conclusiones

- Se analizo la gestión que se aplicara mediante vectores para almacenar registros de pedidos
- Se estudio la gestión y la estructura aplicadas con C++ organizando la información de cada pedido
- Se desarrollo un programa de gestión para una cafetería donde se permita el registro, consultar, modificar, eliminar y generar reportes de pedidos

2.8 Recomendaciones

Utilizar validaciones de datos para evitar errores en la entrada de información y implementar una interfaz más intuitiva al usuario

2.9 Referencias bibliográficas

Ninguna.

2.10 Anexos

Impresión del código:
Menu:

```
===== CAFETERIA =====
1 Registrar pedido
2 Mostrar pedidos
3 Buscar pedido
4 Modificar pedido
5 Eliminar pedido
6 Reporte final
7 Salir
Seleccione opcion:
```

Registrar pedido:

```
===== CAFETERIA =====
1 Registrar pedido
2 Mostrar pedidos
3 Buscar pedido
4 Modificar pedido
5 Eliminar pedido
6 Reporte final
7 Salir
Seleccione opcion: 1
ID: 402
Cliente: kilian
Producto: chocolatesPacari
Cantidad: 4
Precio: 10
```

Mostrar pedido:

```
===== CAFETERIA =====
1 Registrar pedido
2 Mostrar pedidos
3 Buscar pedido
4 Modificar pedido
5 Eliminar pedido
6 Reporte final
7 Salir
Seleccione opcion: 2

ID: 1
Cliente: Ana
Producto: Cafe
Cantidad: 2
Precio: 1.5

ID: 2
Cliente: Luis
Producto: Capuccino
Cantidad: 1
Precio: 2

ID: 3
Cliente: Maria
Producto: Latte
Cantidad: 2
Precio: 2.5

ID: 4
Cliente: Carlos
Producto: Cafe
Cantidad: 3
Precio: 1.5

ID: 5
Cliente: Pedro
Producto: Mocaccino
Cantidad: 1
Precio: 2.8
```

Buscar pedido:

```
===== CAFETERIA =====
1 Registrar pedido
2 Mostrar pedidos
3 Buscar pedido
4 Modificar pedido
5 Eliminar pedido
6 Reporte final
7 Salir
Seleccione opcion: 3
Ingrese ID: 402
Cliente: kilian
```

Modificar pedido:

```
===== CAFETERIA =====
1 Registrar pedido
2 Mostrar pedidos
3 Buscar pedido
4 Modificar pedido
5 Eliminar pedido
6 Reporte final
7 Salir
Seleccione opcion: 4
ID a modificar: 402
Nuevo cliente: yesi
Nuevo producto: fresasconcrema
Nueva cantidad: 1
Nuevo precio: 2
```

Eliminar pedido:

```
Seleccione opcion: 5
ID a eliminar: 1
```

Ya no sale en el registro

```
===== CAFETERIA =====
1 Registrar pedido
2 Mostrar pedidos
3 Buscar pedido
4 Modificar pedido
5 Eliminar pedido
6 Reporte final
7 Salir
Seleccione opcion: 2

ID: 2
Cliente: Luis
Producto: Capuccino
Cantidad: 1
Precio: 2

ID: 3
Cliente: Maria
Producto: Latte
Cantidad: 2
Precio: 2.5
```

Reporte final:

```
6 Reporte final
7 Salir
Seleccione opcion: 6
Total de ventas: 32.3
```

Salir:

```
7 Salir
Seleccione opcion: 7

--------------------------------
Process exited after 511 seconds with return value 0
Presione una tecla para continuar . . .
```